

The Input Problem

A Permanent Bottleneck for Quantum Machine Learning

Muhammad Faryad

Associate Professor of Physics, Lahore University of Management Sciences

maven.com/faryad | The Quantum Edge

2026

Summary. Quantum algorithms are usually presented with their input state handed over for free. When the input is classical data, that convention hides a cost which is often larger than the entire algorithm that follows. This article works through what the three standard encodings actually cost, why the cost is a theorem rather than an engineering problem, and what survives. It accompanies five executable notebooks that reproduce the constructions discussed here and verify every result against a closed-form answer.

1. A convention that hides a cost

Open almost any paper on quantum machine learning and you will find a line of the form: *let $|\psi_x\rangle$ encode the data vector x* . The algorithm then proceeds, and its complexity is quoted in terms of what happens after that line.

The convention is harmless in the settings where it originated. In Shor's algorithm the input is a single integer. In Grover's, it is a black-box oracle whose internal cost is explicitly excluded from the analysis by construction. In both cases nothing is being swept under the rug.

It becomes something else entirely when the input is a length- N vector of real numbers drawn from an actual dataset — a feature vector, a covariance matrix, a probability distribution calibrated to market data. Then the cost of producing $|\psi_x\rangle$ is a cost of the algorithm. And because that cost lives in a preparation stage that no complexity table lists and no circuit diagram displays, it is routinely omitted from exactly the comparisons it should dominate.

This article is about what that cost is. The short version: for generic classical data it is $\Theta(N)$, it is provably so, and it is therefore not a hardware problem that better qubits will eventually solve. Any quantum algorithm consuming such data in $O(\text{polylog } N)$ time is dominated by its own input stage by a factor of $N / \text{polylog } N$.

That is a strong claim, so the rest of this is spent establishing it carefully, and then — equally importantly — bounding it. The input problem does not make quantum machine learning pointless. It makes a particular *style* of quantum machine learning claim untenable, and knowing which style is the difference between building on sand and building on rock.

2. Three ways in, and what each costs

There are three standard encodings. They differ exponentially in qubit count and not at all in asymptotic gate count, which is the first thing worth internalising.

2.1 Basis encoding

Each datum becomes the bit pattern of a computational basis state. A real number is written in fixed point — one sign bit, t_L bits of integer, t_R bits of fraction — and encoded by applying X wherever the bit is 1. A set of M patterns can be placed in equal superposition,

$$|\psi\rangle = \frac{1}{\sqrt{M}} \sum_{i=0}^{M-1} |x^{(i)}\rangle, \quad (1)$$

by the routine of Ventura and Martinez [1]. Two ancilla qubits track which branch is still being processed; at step i , with $M - i$ patterns remaining, a controlled rotation by $\theta_i = 2 \arcsin(1/\sqrt{M-i})$ peels off amplitude

$$\sqrt{\frac{M-i}{M}} \cdot \frac{1}{\sqrt{M-i}} = \frac{1}{\sqrt{M}}, \quad (2)$$

independent of i . The angle varies precisely so that the amplitude does not, and this telescoping is the entire content of the construction.

The cost is $O(t)$ qubits and $O(Mt)$ depth. The depth is not an artefact of the method: M arbitrary t -bit patterns carry $\Theta(Mt)$ bits of information, a circuit of depth d is specified by $O(dt)$ bits, and the prepared state determines the patterns. So $d = \Omega(M)$ for *any* construction. Loading M data points costs at least M steps, and any algorithm running in $o(M)$ has been charged $\Omega(M)$ before it begins.

2.2 Amplitude encoding

The alternative packs 2^n complex numbers into the amplitudes of n qubits,

$$|\psi_x\rangle = \frac{1}{\|\mathbf{x}\|} \sum_{k=0}^{2^n-1} x_k |k\rangle, \quad (3)$$

which is a genuine exponential compression in qubit count and the reason amplitude encoding appears in nearly every quantum machine learning proposal. A 1024-dimensional feature vector fits in ten qubits.

The construction is a binary tree of uniformly controlled rotations [2]. Writing $a_k = |x_k|$ and conditioning on the low i bits reading j ,

$$\theta_{i,j} = 2 \arcsin \sqrt{\mathcal{A}_{i,j} / \mathcal{N}_{i,j}}, \quad \mathcal{N}_{i,j} = \sum_{k \equiv j \pmod{2^i}} a_k^2, \quad \mathcal{A}_{i,j} = \sum_{k \equiv j+2^i \pmod{2^{i+1}}} a_k^2, \quad (4)$$

with a matching cascade of controlled phase gates carrying $\arg x_k$. Summing over levels, $1 + 2 + \dots + 2^{n-1} = 2^n - 1$ angles are required for the magnitudes, and as many again for the phases.

The number to remember is $2^n - 1$. Those angles are computed classically, on an ordinary computer, before a single quantum gate exists. Producing them requires reading the entire data vector. Whatever the quantum circuit does afterwards, the classical preprocessing has already touched every element of the input.

2.3 Distribution loading

The third case is a probability density discretised onto 2^n bins, $|\psi_p\rangle = \sum_k \sqrt{p_k} |k\rangle$. Grover and Rudolph [3] construct it by bisection: at level i , region j spanning $[x_L, x_R)$ with midpoint x_M , define

$$f(i, j) = \frac{\int_{x_L}^{x_M} p(x) dx}{\int_{x_L}^{x_R} p(x) dx}, \quad (5)$$

and rotate by $\theta = \arccos \sqrt{f(i, j)}$. The conditional square roots telescope down the tree to $\sqrt{p_k}$ at the leaves.

This is the construction most often invoked when someone wants to argue that state preparation need not be expensive, and the argument turns on a subtlety that deserves to be stated plainly.

3. Why this is a theorem, not an engineering problem

Lower bound. Preparing an arbitrary $|\psi\rangle \in \mathbb{C}^{2^n}$ from $|0\rangle^{\otimes n}$ requires $\Omega(2^n)$ two-qubit gates.

The argument is a dimension count. The target manifold of normalised states has $2^{n+1} - 2$ real parameters. A circuit of g gates drawn from a fixed two-qubit gate set with continuous parameters spans a manifold of dimension $O(g)$. Covering the target therefore requires $g = \Omega(2^n)$. The bound is met up to constants by known constructions [2, 8, 9], and it is what Qiskit's `StatePreparation` implements.

This is worth dwelling on because it changes the character of the problem. A $\Theta(N)$ gate count is not a limitation of today's compilers, today's qubits, or today's decompositions. Better hardware will not remove it. It is a statement about how much information a circuit can inject per gate, and it will be equally true of a fault-tolerant machine in 2045.

Measurements bear this out with uncomfortable directness. Preparing a smooth cosine distribution, transpiled to the hardware basis $\{cx, rz, sx, x\}$:

n	bins	classical angles	CX (textbook build)	CX (StatePreparation)
3	8	7	64	4
5	32	31	1,180	26
7	128	127	8,300	120
8	256	255	20,180	247

Two lessons sit in that table. The first is practical: a hand-rolled pedagogical decomposition costs roughly eighty times the library implementation, so use the library. The second is structural, and it is the one that matters: look at the last column on its own. It doubles with

every added qubit. StatePreparation is optimal and it is still exponential. The eighty-fold gap is a constant factor; the exponential belongs to both.

And the middle column belongs to neither circuit. No compiler pass will ever touch it.

4. Structure is the escape, and it is narrower than advertised

The lower bound concerns *arbitrary* states. The natural response is to ask what structure buys, and the answer is: a great deal, under a hypothesis that is stronger than it looks.

Return to equation (5). The ratio depends only on the cumulative distribution function evaluated at three points. If the CDF is *efficiently integrable* — computable by a classical circuit of size $\text{poly}(n)$ — then a reversible version of that circuit realises an oracle

$$U_P : |j\rangle |0\rangle \mapsto |j\rangle |f(i, j)\rangle, \quad (6)$$

and a single uniformly controlled rotation per level converts the loaded value into an angle. The cost drops to $O(n \cdot \text{poly}(n))$. This is the actual content of the Grover–Rudolph result and it is a real one.

Now consider what most implementations do instead. They evaluate $f(i, j)$ in Python, obtain $2^n - 1$ numbers, and hard-code each into its own gate. This reproduces the paper’s recursion faithfully, produces exactly the right state, and is exponential. Both versions circulate under the same name, frequently citing the same paper, and the distinction is rarely drawn.

The efficiency lives entirely in the reversible arithmetic that nobody builds. An implementation that computes its angles classically has not implemented an efficient state preparation routine, no matter how faithfully it follows the recursion or how exactly it reproduces the target distribution.

There is a further constraint that is easy to miss. “Efficiently integrable” excludes most densities one actually wants. A lognormal — the distribution underlying Black–Scholes option pricing, and the single most cited application of this machinery — has a CDF with no elementary form.

5. The arithmetic that closes the gap

The clearest demonstration of what the input problem does to a speedup comes from quantum Monte Carlo integration, because there the claimed advantage is precise enough to subtract from.

Quantum amplitude estimation [4] estimates $\mathbb{E}[f(X)]$ to additive error ϵ using $O(1/\epsilon)$ queries to a state-preparation oracle $\mathcal{A}$, against $O(1/\epsilon^2)$ samples for classical Monte Carlo. The quadratic gap is the basis for essentially every proposal in quantum derivative pricing and quantum risk analysis.

Now insert the cost of $\mathcal{A}$. Resolving an expectation to relative precision ϵ requires roughly $N \sim 1/\epsilon$ bins — in the companion notebooks, 32 bins price a European call to within 0.5% of Black–Scholes. By the lower bound, $\mathcal{A}$ costs $\Theta(N)$ gates. Therefore

$$\text{total cost} = \underbrace{O(1/\epsilon)}_{\text{QAE queries}} \times \underbrace{\Theta(1/\epsilon)}_{\text{cost per } \mathcal{A}} = O(1/\epsilon^2), \quad (7)$$

which is the classical scaling. Extrapolating measured gate counts makes it concrete: at $\epsilon = 10^{-5}$ the pipeline needs about 9.6×10^9 two-qubit gates, against 10^{10} classical samples — the same number, one on hardware that does not exist and one on a laptop.

The quadratic advantage was counted in oracle queries under the assumption that the oracle is free. It is not free, and it costs almost exactly enough to close the gap. This is the argument of Herbert [5].

6. Why this bites quantum machine learning hardest

Quantum Monte Carlo is the cleanest illustration, but quantum machine learning is where the input problem is most consequential, for a reason worth spelling out.

The speedups that made QML exciting were exponential and they were *polylogarithmic in the data dimension*. Linear-systems-based proposals promised to manipulate an N -dimensional vector in $O(\text{polylog } N)$ time. That promise is only meaningful if the vector can be got into the machine in comparable time — and by the lower bound, for generic data, it cannot. An input stage costing $\Theta(N)$ against an algorithm costing $\text{polylog}(N)$ is not a constant-factor inconvenience. It is the whole runtime.

The standard escape is to assume a QRAM: an oracle $|i\rangle |0\rangle \rightarrow |i\rangle |x_i\rangle$ delivering data elements in superposition at unit cost. Two observations about that assumption, both due originally to Aaronson’s survey of the fine print [6].

First, no such device exists, and building one requires a hardware architecture qualitatively different from what is being built.

Second — and this is the sharper point — the assumption is not free even in theory. A strong input model that grants efficient superposition access to a structured dataset also grants efficient *classical* sampling access to the same structure. Tang’s dequantization results [7] exploited precisely this: several headline exponential QML speedups collapse to polynomial once the classical algorithm is given input access comparable to what the quantum algorithm was assuming all along. The input assumption that made the quantum algorithm fast made a classical algorithm fast too.

The same pressure appears in kernel methods, where it is less often noticed. A quantum kernel estimates $|\langle \phi(x_i) | \phi(x_j) \rangle|^2$ for pairs of data points, which requires preparing feature states for both. On M training points that is $\Theta(M^2)$ state preparations for the Gram matrix, each with its own circuit and its own shot budget. The feature map may be polynomial-depth and perfectly benign — but the loading is per-data-point, it does not amortise across the dataset, and the quadratic count in M sits underneath every quantum kernel result whether or not the paper reports it.

The pattern is consistent across every branch of QML. Wherever the advantage is claimed against classical data, ask what was assumed about getting that data in. The answer is load-bearing, and it is usually in a sentence rather than a table.

7. What survives

Overstating this would be as careless as ignoring it, so let me be precise about the boundaries. The input problem does not show that quantum machine learning is futile. It shows that one specific argument pattern — classical dataset in, polylogarithmic quantum algorithm,

exponential advantage out — does not close. Several things remain entirely intact.

- **Genuinely efficiently preparable states.** Where a closed-form CDF admits a $\text{poly}(n)$ reversible circuit, Grover–Rudolph works as advertised. These cases are real. They are rarer than the citation pattern suggests, and building the arithmetic is substantial work rather than a footnote.
- **Data the device generates.** If the distribution is the output of a Hamiltonian simulation, a chemistry calculation, or a variationally trained circuit, it was never a classical vector and there is nothing to load. This is the cleanest escape and, not coincidentally, where the strongest near-term results live.
- **Learned loading.** A shallow circuit trained to approximate a distribution — a quantum GAN, for instance [10] — sidesteps the $\Theta(N)$ angle count entirely, because a trained ansatz is not a hard-coded amplitude list. Whether the approximation is good enough for a given downstream task is an open and worthwhile question.
- **Amortisation.** If one loaded state is consumed by many independent measurements, the preparation cost divides. Whether it divides enough is an arithmetic question with a definite answer for any particular pipeline.
- **Polynomial advantages.** Nothing here forbids them. It forbids exponential ones on generic classical input.

8. The lesson

The habit I would most like a reader to take from this is not a fact about state preparation. It is a methodological one, and it generalises well past this topic.

Every construction discussed here is *correct*. Each one was verified against a closed-form answer to fourteen or more significant figures. Correctness was never in question, and correctness told us nothing whatsoever about usefulness. What separated the two was a control experiment: transpile the alternative, count the same quantities, and put both in one table.

That experiment took an afternoon and it inverted the conclusion. It is also, in my experience, the step most often skipped — because a working circuit feels like a result, and running the classical baseline feels like a formality that can wait until a referee asks.

Six questions, before any input-dependent advantage claim.

1. How many numbers are computed classically before the circuit exists? If the answer is $\Theta(N)$, the entire input has already been read.
2. Were the gate counts taken after transpiling to a hardware basis? Abstract counts hide multi-controlled gates completely.
3. Was it benchmarked against the library implementation, or against nothing?
4. Was it benchmarked against the classical algorithm, or against a worse quantum one?
5. Are state error, discretisation error and shot noise reported separately? They routinely differ by ten orders of magnitude, and usually only the smallest is quoted.
6. If the speedup is stated in query complexity — what does the query cost?

Applied to the literature, most papers address two of the six. Applied to your own work before submission, they cost an afternoon and occasionally save a retraction.

The input problem is permanent. It is a counting argument, not a hardware limitation, and it will constrain quantum machine learning for as long as quantum machine learning is applied to classical data. That is not a reason for pessimism — it is a reason to be exact about which claims a given architecture can support. The field will be better served by a smaller number of defensible advantages than by a larger number that dissolve on contact with a gate count.

Companion notebooks

Five executable notebooks reproduce every construction above, verified against Qiskit 2.5.1 and qiskit-aer 0.17.2. Each result is checked against a closed-form answer with an assertion that fails loudly if it drifts.

1. **Basis encoding** — fixed-point conversion and the Ventura–Martinez superposition routine
2. **Amplitude encoding** — the rotation tree, the phase cascade, and a phase-reference bug that is exact on any test vector with a real first component and wrong by $O(1)$ otherwise
3. **Grover–Rudolph loading** — verified bin by bin against exact integrals
4. **Gaussian and lognormal densities** — truncation error, quadrature error, and a European call priced against Black–Scholes
5. **The honest benchmark** — the table in Section 3, and the arithmetic of equation (7)

Available at maven.com/faryad.

References

- [1] D. Ventura and T. Martinez, *Initializing the amplitude distribution of a quantum state*, [arXiv:quant-ph/9807053](https://arxiv.org/abs/quant-ph/9807053) (1998).
- [2] M. Möttönen, J. J. Vartiainen, V. Bergholm and M. M. Salomaa, *Transformation of quantum states using uniformly controlled rotations*, [arXiv:quant-ph/0407010](https://arxiv.org/abs/quant-ph/0407010) (2004).
- [3] L. Grover and T. Rudolph, *Creating superpositions that correspond to efficiently integrable probability distributions*, [arXiv:quant-ph/0208112](https://arxiv.org/abs/quant-ph/0208112) (2002).
- [4] G. Brassard, P. Høyer, M. Mosca and A. Tapp, *Quantum amplitude amplification and estimation*, [arXiv:quant-ph/0005055](https://arxiv.org/abs/quant-ph/0005055) (2000); *Contemp. Math.* **305**, 53 (2002).
- [5] S. Herbert, *No quantum speedup with Grover–Rudolph state preparation for quantum Monte Carlo integration*, *Phys. Rev. E* **103**, 063302 (2021).
- [6] S. Aaronson, *Read the fine print*, *Nature Physics* **11**, 291 (2015).
- [7] E. Tang, *A quantum-inspired classical algorithm for recommendation systems*, *Proc. STOC 2019*, 217 (2019); [arXiv:1807.04271](https://arxiv.org/abs/1807.04271).
- [8] M. Plesch and Č. Brukner, *Quantum-state preparation with universal gate decompositions*, *Phys. Rev. A* **83**, 032302 (2011).

- [9] V. V. Shende, S. S. Bullock and I. L. Markov, *Synthesis of quantum-logic circuits*, IEEE Trans. CAD **25**, 1000 (2006).
- [10] C. Zoufal, A. Lucchi and S. Woerner, *Quantum generative adversarial networks for learning and loading random distributions*, npj Quantum Information **5**, 103 (2019).

A note on the references. The three arXiv identifiers in entries 1–3 are the sources implemented directly in the companion notebooks and are reliable. The remaining entries were compiled from memory during drafting and the volume, page and year details have not been checked against the journals. Please verify before citing. The arguments above rest on the measured gate counts and the dimension-counting bound, both of which stand independently of attribution.